

REGISTRO DE EVALUACIÓN

- NOMBRE: DAVID JOSUE
- APELLIDO: ALONSO JANAMPA
- CÓDIGO: 17200164

• FECHA: 20/06/2025

1. EXAMEN DE DESARROLLO DE SISTEMAS MÓVILES

2. I. ANÁLISIS DE ARQUITECTURA Y CONCURRENCIA

1. EL PATRÓN REPOSITORIO COMO SSOT

RPTA:

El repositorio se considera la única fuente de verdad porque centraliza el acceso a los datos de la aplicación, ya que la UI no consulta directamente a Room ni Retrofit, sino que recibe la información desde el repositorio.

Con respecto al hecho de funcionar como fachada, esto es porque oculta de dónde vienen los datos, puede leer primero desde Room para mostrar datos locales y luego actualizar desde Retrofit si es que hay conexión, si llegan datos nuevos los guarda en Room y la UI se actualiza mediante Flow.

Así es como la interfaz no depende directamente de la base de datos ni de la API, de manera que se reduce el acoplamiento y mejora la mantenibilidad.

2. CORUTINAS Y STRUCTURED CONCURRENCY

RPTA:

viewModelScope es el ámbito de corutinas asociado al ciclo de vida de ViewModel y nos sirve para ejecutar tareas asíncronas, como por ejemplo consultas a base de datos sin bloquear el Main Thread. Esto es importante para la memoria porque las corutinas se cancelan automáticamente cuando ViewModel se destruye, esto evita procesos innecesarios y por eso es mejor usar viewModelScope y no GlobalScope.

3. FLUJOS REACTIVOS (COLD VS HOT)

RPTA:

Un flow es frío porque no emite datos hasta que alguien lo recolecta con collect, es decir, recién va a empezar a trabajar cuando existe un observador activo, en cambio StateFlow es caliente porque mantiene siempre un valor actual en memoria y lo entrega cuando la UI lo observa, además cuando el valor cambia emite automáticamente el nuevo valor.

Para estados de interfaz se prefiere StateFlow porque la UI siempre recibe el estado actual, también se integra mejor con corutinas, Flow y Compose de manera que la pantalla se recompone cuando cambia el estado.

4. INYECCIÓN DE DEPENDENCIAS (DI) MANUAL

NOTA:

La DI Manual consiste en crear las dependencias principales dentro de una clase que extiende de Application, así ya se pueden inicializar objetos como la base de datos, los DAO y el repositorio, esto permite que varios componentes de la app usen la misma instancia del repositorio, así no se crean objetos repetidos y se mantiene una estructura más ordenada.

by lazy sirve para que esos objetos se creen recién cuando se usan por primera vez, así se ahorran recursos y asegura que la base de datos o repositorio se inicialicen una vez solamente.

3. II SECCIÓN DE COMPLETAR

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia, se implementa usando el tipo Flow
2. El operador Dispatchers.Default permite ejecutar tareas pesadas fuera del Main Thread específicamente optimizado para tareas de CPU como el parseo masivo de datos.
3. Para que un servicio en primer plano de ubicación sea válido en Android 10+ debe declararse en el manifiesto con el tipo android:foregroundServiceType = "location"
4. La función applicationContext de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un cast.
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama SuspendCancellableCoroutine
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama AndroidManifest.xml
7. En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL
8. El operador de Flow que permite combinar múltiples fuentes como GPS y Sensores para emitir un estado unificado es combine()

4. III DESARROLLO DE CÓDIGO Y LÓGICA PERSONALIZADA

1. Repositorio y DI

```
import android.app.Application

class UsuarioRepository {
    private val usuarioDao

    class UsuarioApplication : Application() {

        val database by lazy {
            AppDatabase.getDatabase(this)
        }

        val usuarioRepository by lazy {
            UsuarioRepository(database, usuarioDao())
        }
    }
}
```

2. Lógica dinámica de Identidad

```
data class Usuario (
    val nombre : String,
    val apellidos : String,
    val edad : Int?
)

fun main() {
    val usuario = Usuario (
        nombre = "David",
        apellidos = "Huatuco Juampa",
        edad = null
    )

    val nombre : String? = usuario.nombre
    val apellidos : String? = usuario.apellidos

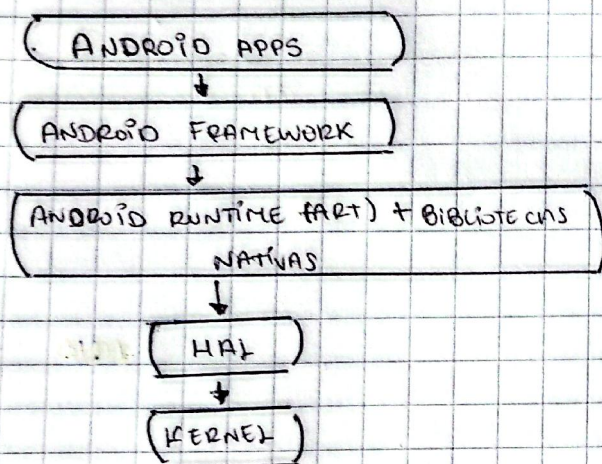
    val puntaje = (nombre?.replace(".", "")?.length?:0) + (apellidos?.replace(".", "")?.length?:0)

    val nombreCompleto = "${nombre?: ""} ${apellidos?: ""}"

    println("Usuario: $nombreCompleto, tu puntaje de Identidad es: $puntaje")
}
```

3. IV ARQUITECTURA DEL AOSP COMPLETA

1. Gráfico



2. IDENTIFICACIÓN TOTAL

1. **ANDROİD APPS**: Contiene las aplicaciones que usa el usuario o que vienen ya instaladas en el sistema.
2. **ANDROİD FRAMEWORK**: Contiene las clases API's y servicios que permiten a las apps comunicarse con el sistema sin acceder directamente al hardware.
3. **ART + BIBLIOTECAS NATIVAS**: ART ejecuta apps convirtiendo bytecode en instrucciones para el procesador, también están las bibliotecas nativas como libc, libbinder y otros componentes nativos.
4. **HAL**: Es la capa de abstracción de hardware que funciona como puente entre android y los controladores del dispositivo como cámara, GPS, audio, etc.
5. **KERNEL**: Linux Kernel es el núcleo del sistema operativo administra procesos, memoria, seguridad, energía y se comunica directamente con el hardware.